

**A
Project Report
On
CitiCare**

Prepared by

Apurva Jayswal 23DCE045
Mahipal Jinjala 23DCE046
Purv Kapuriya 23DCE055
Ayush Vekariya 23DCE127

Under the guidance of

Mr. Prashant Jadav, Assistant Professor,
CE Department,
DEPSTAR, CHARUSAT

Submitted to

Charotar University of Science & Technology
Degree of Bachelor of Technology
in Computer Engineering
CE367: Project - IV
of 6th Semester of B.Tech

Submitted at



DEPARTMENT OF COMPUTER ENGINEERING

Faculty of Technology & Engineering, CHARUSAT

Devang Patel Institute of Advance Technology and Research (DEPSTAR)

At: Changa, Dist: Anand – 388421

April 2026

CERTIFICATE

This is to certify that the report entitled “**CitiCare**” is a bonafied work carried out by **Purv Kapuriya 23DCE055** under the guidance and supervision of Prof. Prashant Jadav for the subject **CE367 PROJECT- IV** of 6th Semester of Bachelor of Technology in **Computer Engineering** at Devang Patel Institute of Advance Technology and Research (DEPSTAR), Faculty of Technology & Engineering (FTE) – CHARUSAT, Gujarat.

To the best of my knowledge and belief, this work embodies the work of candidate himself, has duly been completed, and fulfills the requirement of the ordinance relating to the B.Tech. Degree of the University and is up to the standard in respect of content, presentation and language for being referred to the examiner.

Under the supervision of,

Mr. Prashant Jadav,
Assistant Professor,
CE Department,
DEPSTAR, CHARUSAT-Changa.

Dr. Chirag Patel
Head of Department,
Dept. of Computer Engineering,
DEPSTAR, CHARUSAT-
Changa.

Devang Patel Institute of Advance Technology and Research (DEPSTAR)

Faculty of Technology & Engineering, CHARUSAT

At: Changa, Ta. Petlad, Dist. Anand, PIN: 388 421. Gujarat

DECLARATION BY THE CANDIDATES

We hereby declare that the project report entitled “**CitiCare**” – Smart Citizen Complaint Management System” submitted by us to Devang Patel Institute of Advance Technology and Research (DEPSTAR), Changa, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Engineering, is a record of bonafide work carried out by us under the guidance of **Mr. Prashant Jadav**.

We further declare that the work carried out and documented in this project report has not been submitted elsewhere either in part or in full for the award of any other degree or diploma in this institute or any other institute or university. This project is the result of our own efforts and has been completed in accordance with the academic guidelines provided.

Mr. Apurva Jayswal (23DCE045)
Mr. Mahipal Jinjala (23DCE046)
Mr. Purv Kapuriya (23DCE055)
Mr. Ayush Vekariya (23DCE127)

This is to certify that the above statement made by the candidates is correct to the best of my knowledge.

Signature of Guide: _____

TABLE OF CONTENTS

LIST OF FIGURES.....	4
ABSTRACT.....	5
CHAPTER 1: INTRODUCTION.....	6
1.1 Background of the Project.....	6
1.2 Problem Definition.....	6
1.3 Motivation for the Project.....	6
1.4 Objectives and Scope of the Project.....	6
CHAPTER 2: LITERATURE REVIEW.....	8
2.1 Research and existing solutions in the domain.....	8
2.2 Comparative analysis of existing platforms.....	8
2.3 How the project differs from or builds upon existing work.....	8
CHAPTER 3: SYSTEM ANALYSIS.....	9
3.1 Functional Requirements.....	9
3.2 Non-Functional Requirements.....	9
CHAPTER 4: TECHNOLOGY STACK.....	10
4.1 List of languages, frameworks, and tools used.....	10
4.2 Explanation of why these tools were selected.....	10
CHAPTER 5: SYSTEM DESIGN.....	11
5.1 Use Case Diagram.....	11
5.2 Architecture Diagram.....	11
5.3 Database Design.....	12
5.4 UI/UX Design.....	12
5.5 Modules/Components Overview.....	13
5.6 Features Developed.....	13
CHAPTER 6: TESTING.....	14
6.1 Types of testing performed.....	14
6.2 Tools used for testing.....	14
6.3 Test Cases and Results.....	14
CHAPTER 7: RESULTS.....	15
7.1 Screenshots of the final product.....	15
7.2 Explanation of functionality achieved.....	18

CHAPTER 8: CHALLENGES FACED.....	19
8.1 Development challenges encountered.....	19
8.2 Solutions or workarounds used.....	19
CHAPTER 9: CONCLUSION AND FUTURE SCOPE.....	20
9.1 Summary of the project achievements.....	20
9.2 Potential areas for improvement or future features.....	20
REFERENCES.....	21

LIST OF FIGURES

5.1 Use Case Diagram.....	13
5.2 Architecture Diagram.....	14
5.3 Database Design.....	15
7.1 Landing Page.....	19
7.2 User Dashboard.....	19
7.3 User File Complaint.....	19
7.4 User Complaints.....	20
7.5 Other's Complaints.....	20
7.6 View Complaint with Status.....	20
7.7 Department Dashboard.....	21
7.8 Admin Pannel.....	21
7.9 User Management.....	21
7.10 Admin Complaint Dashboard.....	22
7.11 All Departments.....	22
7.12 Complaints Analytics.....	22

ABSTRACT

CitiCare – Smart Citizen Complaint Management System is a full-stack web application designed to improve communication and coordination between citizens and municipal authorities by providing a centralized digital platform for reporting civic issues such as potholes, water leakage, garbage collection problems, and street light failures. The system overcomes the limitations of traditional manual complaint systems by allowing users to submit complaints with descriptions, location details, and supporting images or videos, while also enabling citizens to view and upvote existing complaints to help authorities prioritize critical issues. The platform includes multiple modules such as secure user authentication, complaint management, department dashboards, budget estimation, and administrative control, where departments can track complaints, update their status, upload work completion proof, and request budget approvals, while the Municipal Head (Admin) monitors complaints, approves budgets, and analyzes system performance through dashboards. Developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js), the system ensures scalability, flexibility, and efficient data management, while additional features like map-based location tracking and real-time status updates enhance usability. Overall, CitiCare provides a smart, transparent, and efficient solution for urban issue management, promoting better governance and stronger citizen engagement.

INTRODUCTION

1.1 Background of the Project

In modern urban environments, the management of civic issues such as potholes, water leakage, garbage collection, and infrastructure maintenance remains a significant challenge for municipal authorities. Traditional complaint systems are often manual, time-consuming, and lack proper tracking mechanisms, resulting in delayed responses and lack of transparency.

With the rapid advancement of digital technologies, there is a growing need for smart solutions that can bridge the communication gap between citizens and government authorities. Citizens require a platform where they can easily report issues and track their resolution status in real-time.

The **CitiCare – Smart Citizen Complaint Management System** is developed as a web-based solution to address these challenges. It enables citizens to submit complaints digitally, attach relevant media, and track progress, while allowing departments to manage and resolve issues efficiently.

1.2 Problem Definition

Existing municipal complaint systems suffer from several limitations:

- Lack of a centralized digital platform for complaint registration
- Poor communication between citizens and authorities
- No transparency in complaint status tracking
- Delayed resolution of civic issues
- No prioritization mechanism for critical problems
- Difficulty in managing complaints across multiple departments

Due to these issues, citizens often face inconvenience and dissatisfaction with public services.

The problem is to design and develop a system that ensures **efficient complaint management, transparency, and accountability** in handling civic issues.

1.3 Motivation for the Project

The motivation behind developing CitiCare is to create a **smart and efficient system** that improves the quality of urban governance and citizen engagement.

Key motivating factors include:

- Improving communication between citizens and municipal authorities
- Providing transparency in complaint handling
- Enabling faster resolution of civic issues
- Reducing manual effort and paperwork
- Introducing a priority-based system using upvotes
- Supporting the vision of **Smart Cities and Digital India initiatives**

This project aims to make city management more responsive and citizen-friendly.

1.4 Objectives and Scope of the Project

Objectives

The main objectives of CitiCare are:

- To develop a centralized platform for reporting civic issues
- To allow users to submit complaints with location and media support
- To enable departments to track and resolve complaints efficiently
- To provide real-time status updates to users
- To implement a priority system using complaint upvotes
- To allow administrative control for monitoring and budget approval

Scope of the Project:

The scope of the CitiCare system includes:

- Citizen interface for complaint submission and tracking
- Department dashboards for complaint handling and updates
- Admin dashboard for monitoring and decision-making
- Integration of map-based location services
- Image upload and status tracking features
- Basic analytics and reporting functionalities

The system is designed as a scalable web application that can be extended with additional features such as mobile applications, AI-based issue detection, and real-time notifications in the future.

LITERATURE REVIEW

2.1 Research and existing solutions in the domain

In recent years, several digital platforms have been developed to address civic issues and improve communication between citizens and government authorities. Many municipal corporations have introduced online complaint systems and mobile applications to streamline public service management.

Some notable existing systems include:

- **Municipal Corporation Portals:** Many cities provide official websites where citizens can register complaints related to public services such as roads, sanitation, and water supply.
- **Mobile Applications (e.g., Smart City Apps):** Applications like Swachh Bharat App and local municipal apps allow users to report issues with images and location details.
- **Social Media Platforms:** Citizens often use platforms like Twitter or Facebook to raise complaints, tagging government authorities for quick action.

While these systems have improved accessibility, they still face several limitations in terms of efficiency, transparency, and user engagement.

2.2 Comparative analysis of existing platforms

Feature	Traditional System	Existing Digital Platforms	CitiCare System
Complaint Registration	Manual	Online forms/apps	Easy web interface
Status Tracking	Limited	Available but unclear	Real-time tracking
Media Upload	Not supported	Supported	Fully supported
Location Tracking	Not available	Basic	Map-based integration
Priority System	Not available	Limited	Upvote-based priority
Department Handling	Manual routing	Semi-automated	Fully structured
Transparency	Low	Moderate	High

2.3 Explain how your project differs from or builds upon existing work

The **CitiCare system** is designed to overcome the limitations of existing platforms by introducing several advanced and user-friendly features:

- **Centralized Platform:** Provides a single platform for all types of civic complaints.
- **Upvote-Based Priority System:** Allows citizens to upvote issues, helping authorities identify high-priority problems.
- **Real-Time Status Updates:** Ensures transparency by allowing users to track complaint progress.

- **Department-Specific Dashboards:** Enables efficient handling of complaints by respective departments.
- **Admin Control System:** Allows municipal heads to monitor complaints and approve budgets.
- **Map-Based Location Integration:** Improves accuracy of complaint location using geographic coordinates.
- **Media Support:** Allows users to upload images/videos for better understanding of issues.

Unlike traditional and existing systems, CitiCare focuses on **efficiency, transparency, automation, and user engagement**, making it a more effective solution for modern urban governance.

SYSTEM ANALYSIS

3.1 Functional Requirements

Functional requirements describe the core functionalities that the system must perform. The CitiCare system includes multiple user roles such as Citizen, Department Admin, and Municipal Head, each with specific functionalities.

1. Citizen Module

- User registration and login
- Submit complaints with:
 - Title and description
 - Category selection
 - Location details (address, map coordinates)
 - Image/video upload
- View all complaints
- Filter complaints based on:
 - Area
 - Department
 - Status
- Upvote existing complaints
- Track complaint status (Pending, Processing, Solved)

2. Department Module

- Department login authentication
- View complaints assigned to their department
- Access detailed complaint information
- Accept or reject complaints
- Update complaint status:
 - Pending → Accepted → Processing → Solved
- Upload before and after images of completed work
- Add remarks for complaint resolution
- Submit cost estimation for complaint resolution

3. Municipal Head (Admin) Module

- Admin login and secure access
- View all complaints across departments
- Monitor complaint status and progress
- Review and approve/reject budget requests
- Analyze complaint statistics:
 - Total complaints

- Resolved vs pending
- Department-wise distribution

4. System Functionalities

- Role-based authentication and authorization
- Complaint management system
- Image upload and storage
- Location tracking using map integration
- Upvote-based priority system
- Notification of status updates (basic)

3.2 Non-Functional Requirements

Non-functional requirements define the quality and performance aspects of the system.

1. Performance Requirements

- The system should handle multiple users simultaneously
- Fast response time for API requests
- Efficient data retrieval from the database

2. Security Requirements

- Secure authentication using JWT
- Password encryption using hashing (bcrypt)
- Protection of sensitive data using environment variables
- Role-based access control

3. Usability Requirements

- Simple and user-friendly interface
- Easy complaint submission process
- Clear navigation for all dashboards

4. Reliability Requirements

- System should be available with minimal downtime
- Proper error handling and validation
- Data consistency in database operations

5. Scalability Requirements

- System should support increasing number of users and complaints
- Modular backend design for easy expansion
- Ability to add new features in future

6. Compatibility Requirements

- Web application should run on all modern browsers
- Responsive design for different screen sizes

TECHNOLOGY STACK

4.1 List of languages, frameworks, and tools used

The CitiCare system is developed using modern web technologies to ensure scalability, performance, and efficient communication between users and the system. The following table summarizes the technologies used:

Category	Technology	Purpose
Frontend	React.js	Building user interface and dashboards
Frontend	HTML5	Structuring web pages
Frontend	CSS3	Styling the application
Frontend	Tailwind CSS	Responsive and modern UI design
Frontend	JavaScript	Client-side scripting
Backend	Node.js	Server-side runtime environment
Backend	Express.js	Building RESTful APIs
Database	MongoDB	Storing application data
Database	Mongoose	Object Data Modeling (ODM)
Authentication	JWT	Secure user authentication
Security	bcrypt	Password hashing
File Handling	Multer	Uploading images/videos
API Handling	Axios	Connecting frontend with backend
Version Control	Git	Tracking code changes
Repository	GitHub	Hosting project code
Testing Tool	Postman	API testing
Debugging Tool	Chrome DevTools	Debugging frontend and network

4.2 Explanation of why these tools were selected

1. React.js

React.js is a JavaScript library used for building dynamic and interactive user interfaces. It allows the creation of reusable components and efficient rendering using the virtual DOM.

2. Node.js

Node.js is a server-side runtime environment that allows JavaScript to run outside the browser. It is used to handle backend operations and API requests.

3. Express.js

Express.js is a lightweight framework for Node.js used to build APIs and manage routing and middleware efficiently.

4. MongoDB

MongoDB is a NoSQL database that stores data in a flexible JSON-like format. It is used to manage user data, complaints, and system records.

5. Mongoose

Mongoose provides a structured way to define schemas and interact with MongoDB, making database operations easier and more efficient.

6. JWT (JSON Web Token)

JWT is used for authentication and authorization, ensuring secure access to system resources.

7. bcrypt

bcrypt is used to hash passwords before storing them in the database, enhancing security.

8. Multer

Multer is middleware used for handling file uploads, allowing users to upload complaint images.

9. Axios

Axios is used to send HTTP requests from the frontend to the backend APIs.

10. Git & GitHub

Git is used for version control, while GitHub is used for storing and managing the project repository.

11. Postman

Postman is used for testing APIs to ensure correct request-response behavior.

12. Chrome DevTools

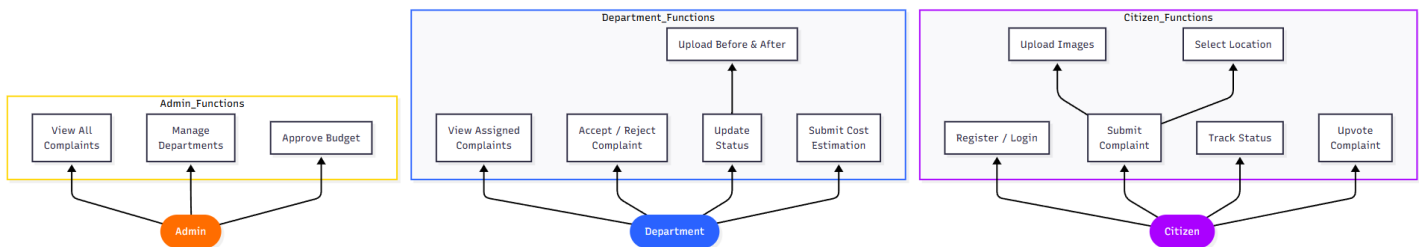
Chrome DevTools helps in debugging frontend issues and analyzing network performance.

SYSTEM DESIGN

5.1 Use Case Diagram

The system supports two primary actors: **Student** (Registered User) and **Admin**.

- **Guest User** can: Register, Login.
- **Student** can:
 - **Manage Profile** (Update Info, Change Password).
 - **Track Progress** (View Dashboard, Update Progress).
 - **Access Modules** (Study Materials, Quizzes, Coding Practice, Interview Prep, Resume Analyzer).
- **Admin** can:
 - **Manage Users** (View all users, Update Roles, Deactivate Accounts).

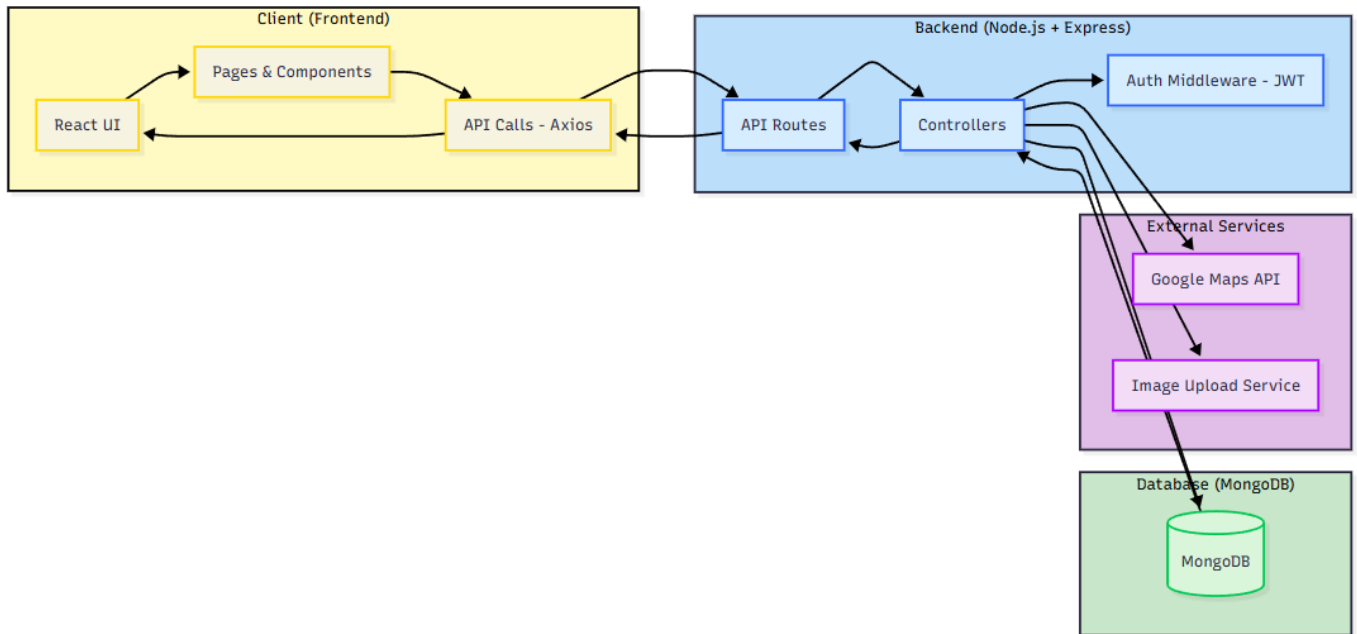


5.1 Use Case Diagram

5.2 Architecture Diagram

SmartPrep utilizes a **Client-Server Architecture** following the Monolithic MERN pattern:

1. **Client (Frontend):** **React.js** application runs in the browser, handling all user interactions and rendering with routing via react-router-dom.
2. **API Layer (Backend):** **Express.js** handles all business logic, manages JWT-based **authentication**, and routes API calls (e.g., /api/auth, /api/users, /api/resume).
3. **Data Layer (Database):** **MongoDB** persists user data, profiles, progress metrics, and analysis reports.
4. **External Services:** The backend communicates with an AI Inference API (via @gradio/client) for the complex task of resume analysis and scoring.



5.2 Architecture Diagram

5.3 Database Design

The database design in MongoDB consists of two main collections:

users:

`_id`: ObjectId (Primary Key)

`name`: String

`email`: String (Unique)

`password`: String (Hashed)

`role`: String ('citizen', 'department', 'admin')

`phone`: String

`address`: String

`createdAt`: Date

complaints:

`_id`: ObjectId (Primary Key)

`title`: String

`description`: String

`category`: String ('Pothole', 'Water Leakage', 'Garbage', 'Street Light', 'Sewage', 'Encroachment')

`status`: String ('Pending', 'Accepted', 'Processing', 'Solved', 'Rejected')

`userId`: ObjectId (Foreign Key to users)

`departmentId`: ObjectId (Foreign Key to departments)

`location`: {

latitude: Number
longitude: Number
address: String

}

images: Array of Strings (image URLs)

upvotesCount: Number

createdAt: Date

updatedAt: Date

departments:

_id: ObjectId (Primary Key)

name: String ('Water', 'Road', 'Electricity', 'Sanitation')

description: String

work_updates:

_id: ObjectId (Primary Key)

complaintId: ObjectId (Foreign Key to complaints)

beforeImage: String

afterImage: String

remarks: String

updatedBy: ObjectId (Foreign Key to users - department)

updatedAt: Date

votes:

_id: ObjectId (Primary Key)

userId: ObjectId (Foreign Key to users)

complaintId: ObjectId (Foreign Key to complaints)

createdAt: Date

admin:

_id: ObjectId (Primary Key)

name: String

email: String (Unique)

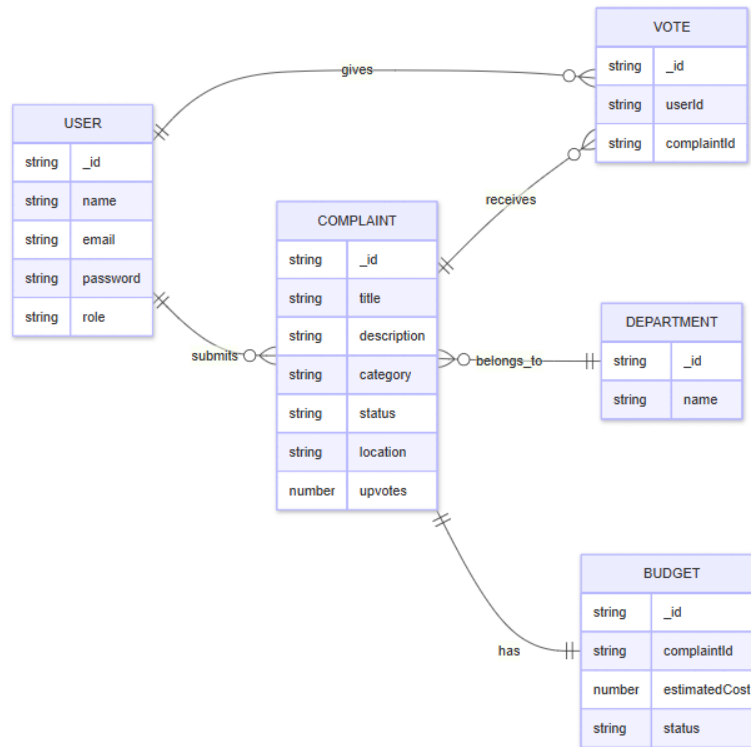
password: String (Hashed)

role: String ('admin')

phone: String

designation: String

createdAt: Date



5.3 Database Design

5.4 UI/UX Design

The application's design prioritizes a **clean, functional, and modern aesthetic** using React components styled with **Tailwind CSS** and **Shadcn UI**.

- **Color Palette:** Employs a primary color set around blue (--primary: 220 100% 50%) for key interactive elements and success indicators in green.
- **Navigation:** A responsive **Navbar** provides access to all core modules.
- **Dashboard:** The central hub uses **Card** components to display key performance indicators and a visual **Progress** bar for overall completion.
- **Module Layout:** Dedicated pages for complex interactions (e.g., a split-screen layout on the CodingPractice page for problem/editor).
- **Notifications:** Uses the useToast hook for status feedback on actions like login, code execution, and test completion.

5.5 Modules/Components Overview

Architecture:

Frontend: React, Vite, TailwindCSS, Shadcn/ui

Backend: Node.js, Express, MongoDB

AI & Services: Hugging Face (Resume Analysis), Judge0 (Code Execution)

Frontend Modules:

- **AuthContext** and **ProtectedRoute** – JWT-based authentication and role-based routing

- Core Pages:
 - LandingPage (public)
 - Login / Register
 - Student Dashboard (StudyMaterials, Quizzes, CodingPractice, InterviewPrep, ResumeAnalyzer, Profile)
 - Admin Panel (User & Content Management)
- UI Library: Reusable layout, form, data, and feedback components via Shadcn/ui

Backend Modules:

- Models: User, Theory, CodingProblem, ResumeAnalysis, StudyMaterial, Submission
- Controllers: Handle authentication, user data, resume AI processing, and content management
- Routes: Organized by module (auth, users, resume, theory, coding-problems, uploads)
- Middleware: JWT authentication, input validation, rate limiting, CORS, Helmet security

Key Functional Modules:

1. Authentication – Secure login/register, JWT, role-based access
2. Study Materials – DSA topics, categorized content, progress tracking
3. Coding Practice – Problem sets with Judge0 integration for live execution
4. Resume Analysis – AI-powered analysis with scoring, suggestions, and keyword insights
5. Quizzes – Interactive tests with performance tracking
6. Admin Panel – Manage users, content, and system analytics

Integrations and Security:

- External: MongoDB, Hugging Face, Judge0
- Security: Rate limiting, Helmet, CORS, validation, JWT token protection

Data Flow:

User → Auth → API → Database → Frontend Display

Key pipelines:

- Login/Register → Token → Dashboard
- Theory & Quizzes → Models → UI
- Resume Upload → AI Analysis → Feedback Display
- Code Submission → Judge0 → Results

5.6 Features Developed

- **JWT Authentication & Authorization:** Secure user session management and explicit role-based checks for Admin access to user management routes.
- **Personalized Progress Tracking:** The system tracks and displays completed quizzes, solved coding problems, and study materials read.

- **File-based Resume Analysis:** Enables users to upload PDF, DOCX, or TXT resumes for text extraction and scoring.
- **Simulated Code Editor:** Provides a functional interface for code submission, clearly displaying inputs, expected outputs, and test case results.
- **Multi-Category Quizzes:** A timed assessment module with question navigation, answering, and results breakdown for DSA, OS, Aptitude, etc..

TESTING

6.1 Types of testing performed

Testing is an essential phase in software development to ensure that the system functions correctly and meets user requirements. The following types of testing were performed for the CitiCare system:

1. Unit Testing

Each individual module of the system, such as authentication, complaint creation, and API endpoints, was tested separately to ensure correct functionality.

2. Integration Testing

Integration testing was performed to verify that different modules (frontend, backend, and database) work together properly. For example, testing whether complaint data submitted from the frontend is correctly stored in the database.

3. Functional Testing

Functional testing ensured that all features of the system work according to requirements. This includes complaint submission, status updates, upvoting, and dashboard functionalities.

4. API Testing

All backend APIs were tested using tools like Postman to ensure correct request handling, response structure, and error handling.

5. User Interface (UI) Testing

UI testing was performed to ensure that all pages are properly displayed, responsive, and user-friendly across different screen sizes.

6. System Testing

Complete system testing was carried out to verify the end-to-end functionality of the application, including all user roles (Citizen, Department, Admin).

6.2 Tools used for testing

The following tools were used during the testing phase:

Tool	Purpose
Postman	Testing backend APIs

Tool	Purpose
Google Chrome DevTools	Debugging frontend and network requests
Browser Testing	Verifying UI and responsiveness
MongoDB Compass	Verifying database data

6.3 Test Cases and Results

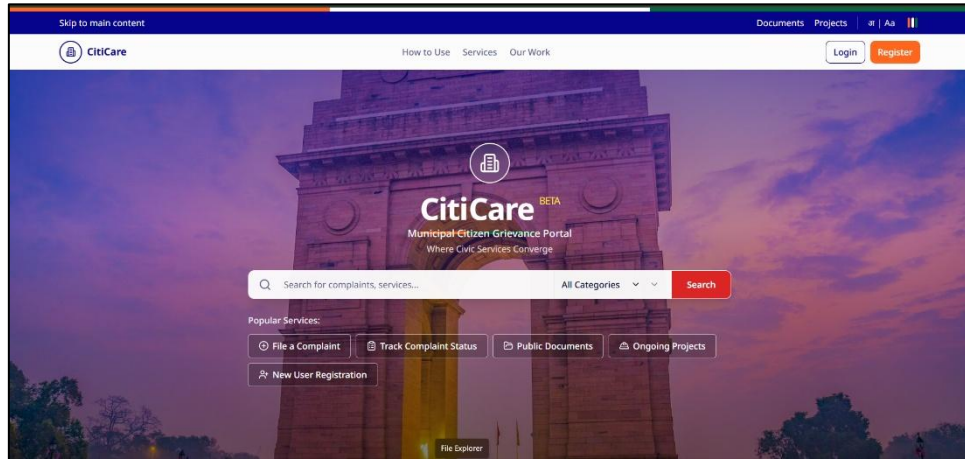
Below are some sample test cases used to validate the system:

Test Case Table

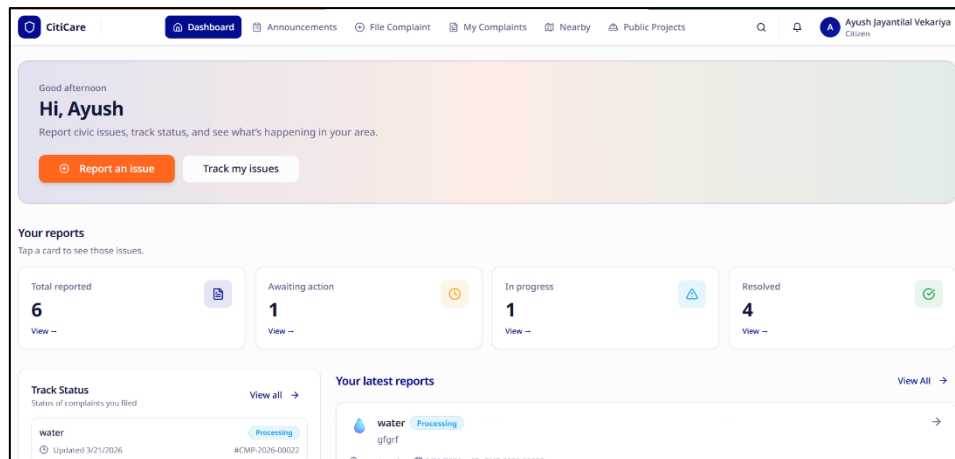
Test Case ID	Test Description	Input	Expected Output	Result
TC01	User Registration	Valid user data	User registered successfully	Pass
TC02	User Login	Correct credentials	Login successful	Pass
TC03	Complaint Submission	Valid complaint data	Complaint stored in database	Pass
TC04	Image Upload	Upload image file	Image saved successfully	Pass
TC05	Complaint Listing	Fetch complaints	Display complaint list	Pass
TC06	Upvote Complaint	Click upvote	Vote count increases	Pass
TC07	Status Update	Change status	Status updated correctly	Pass
TC08	Budget Request	Submit cost	Request stored	Pass
TC09	Admin Approval	Approve request	Status updated to approved	Pass
TC10	Map Location	Select location	Coordinates stored	Pass

RESULT

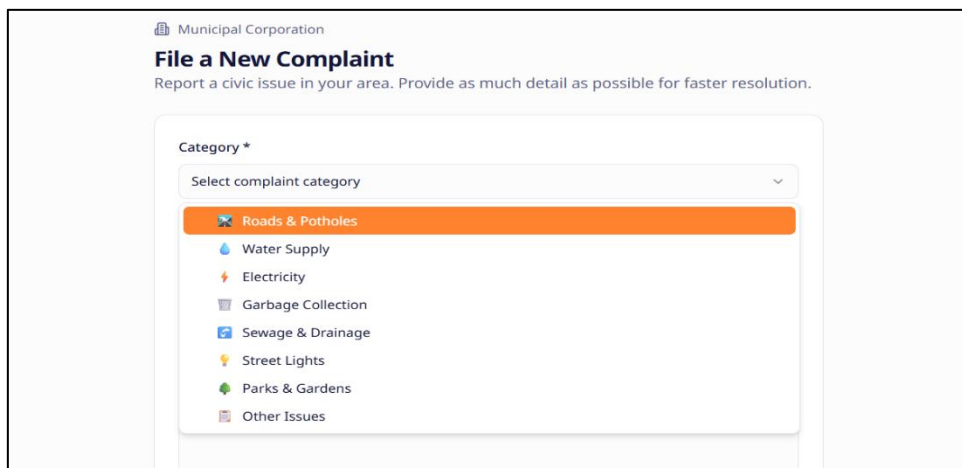
7.1 Screenshots of the final product



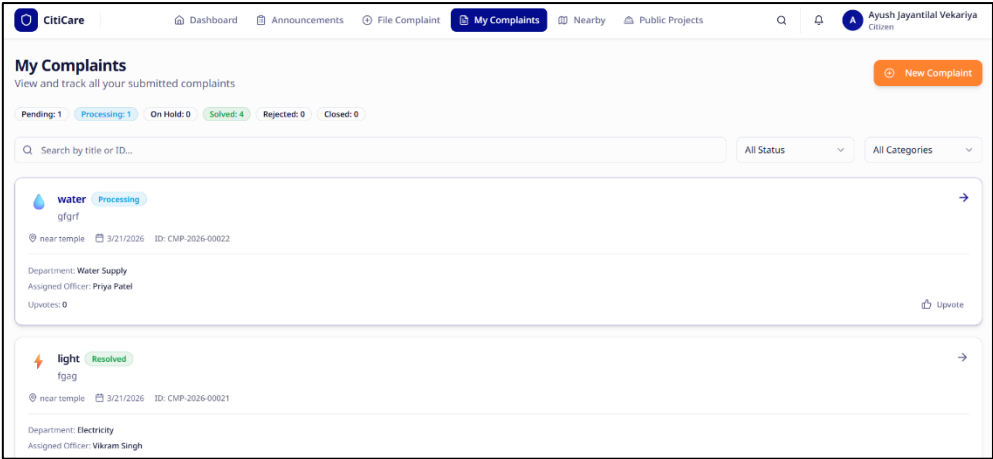
7.1 Landing Page



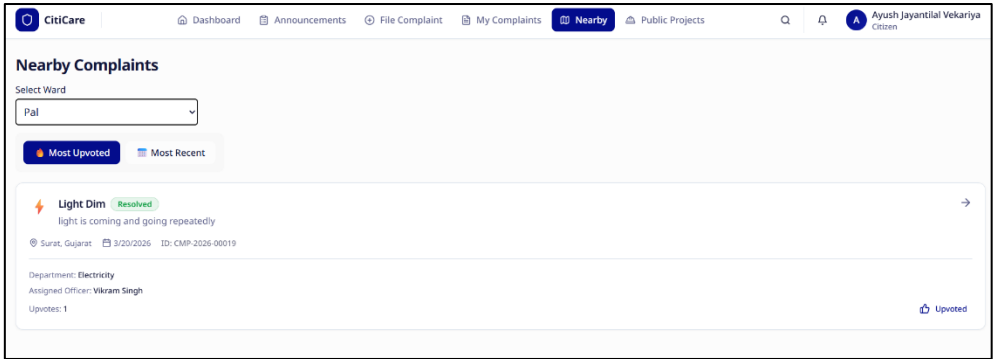
7.2 User Dashboard



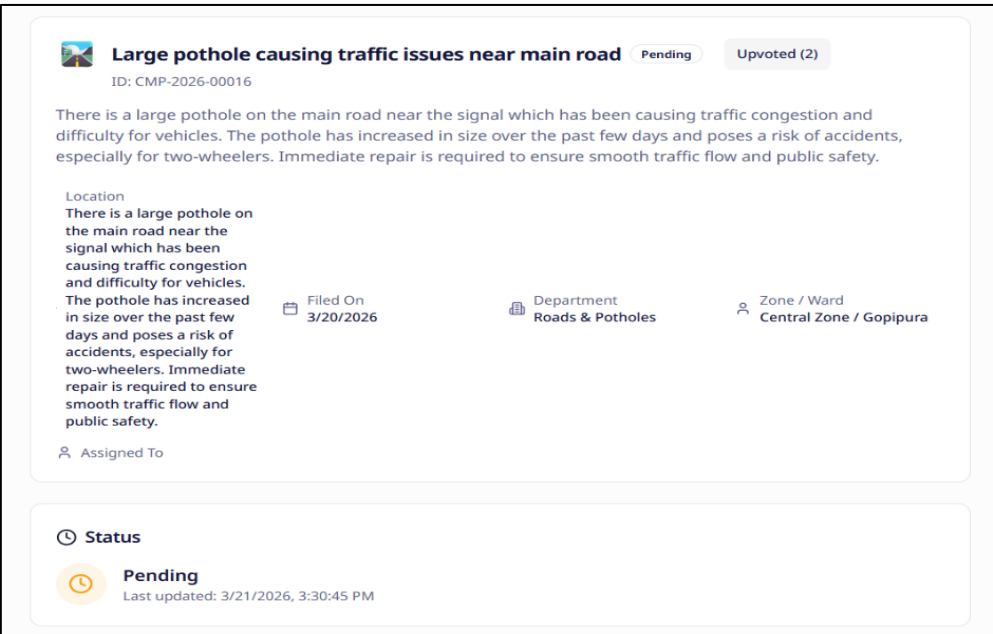
7.3 User File Complaint



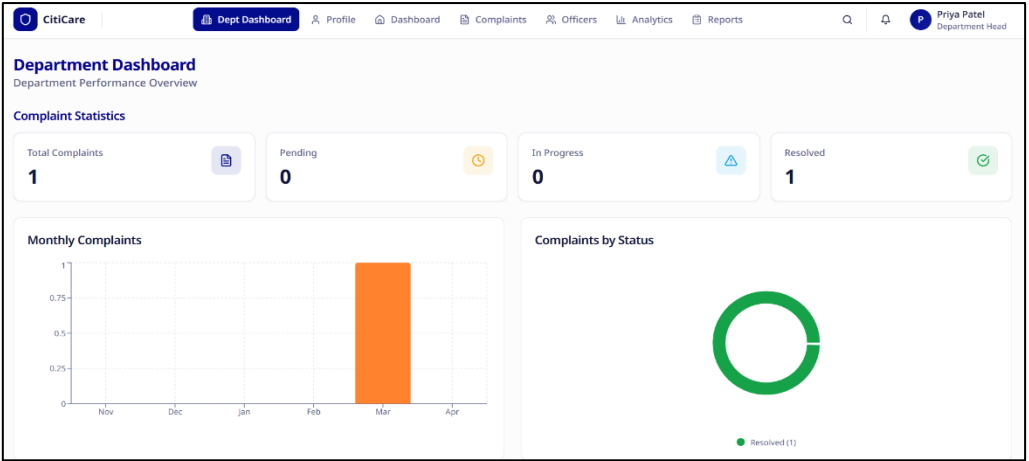
7.4 User Complaints



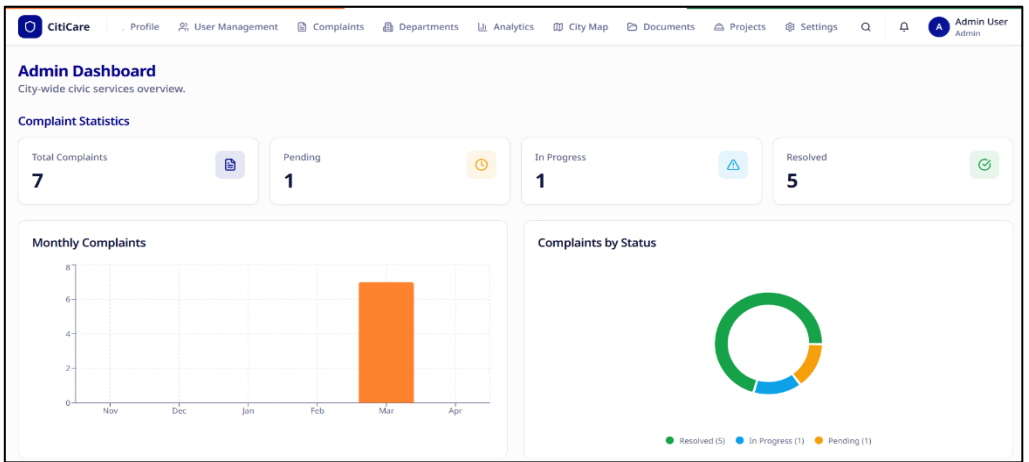
7.5 Other's Complaints



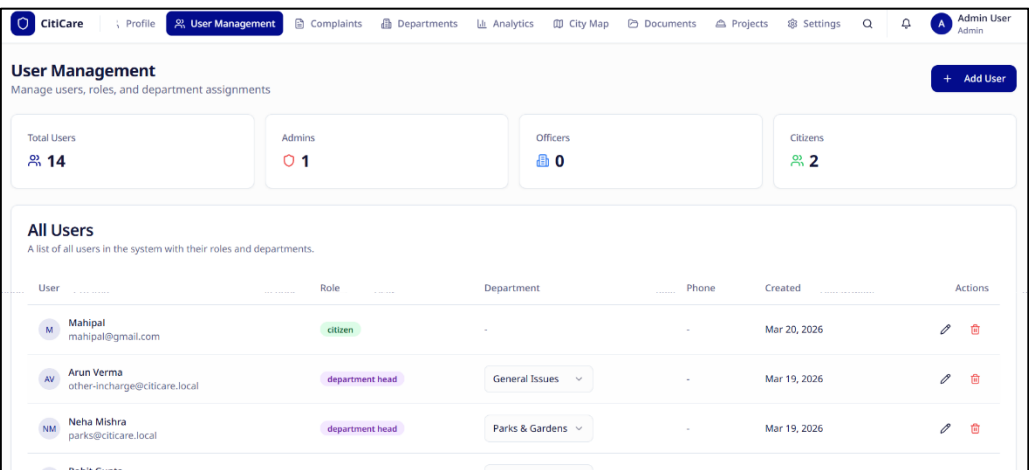
7.6 View Complaint with Status



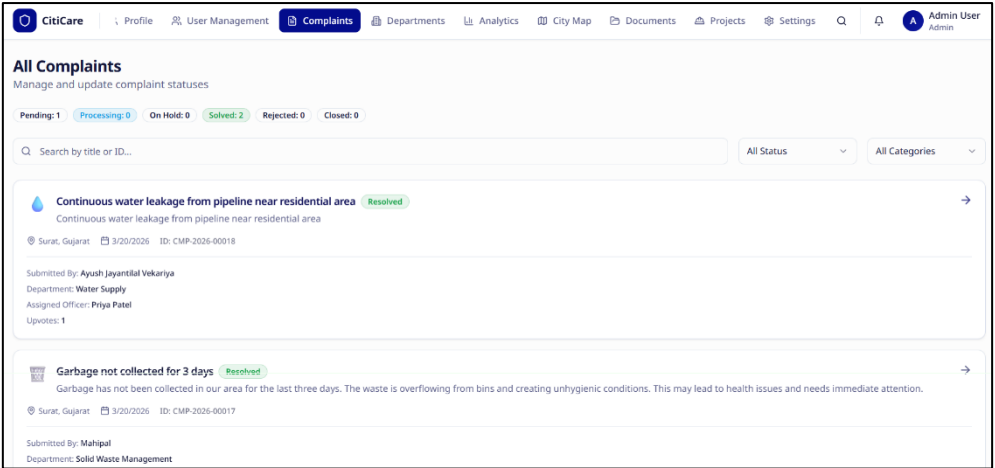
7.7 Department Dashboard



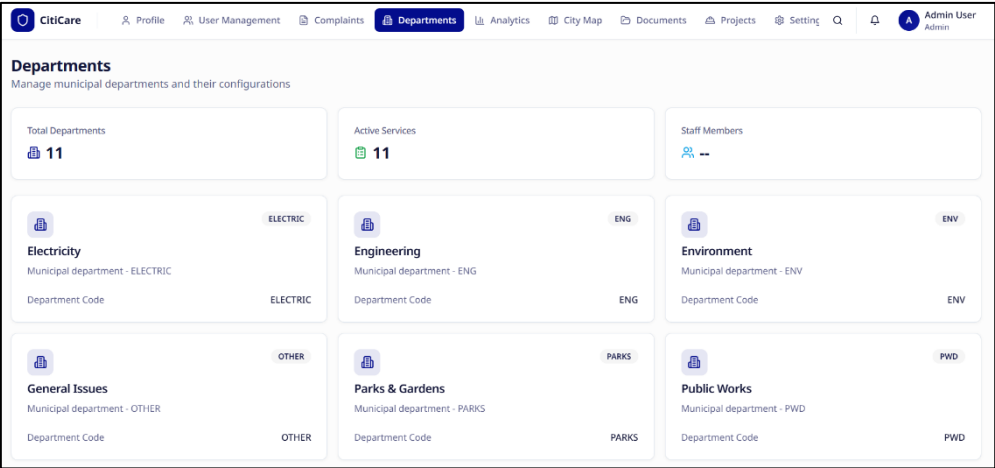
7.8 Admin Pannel



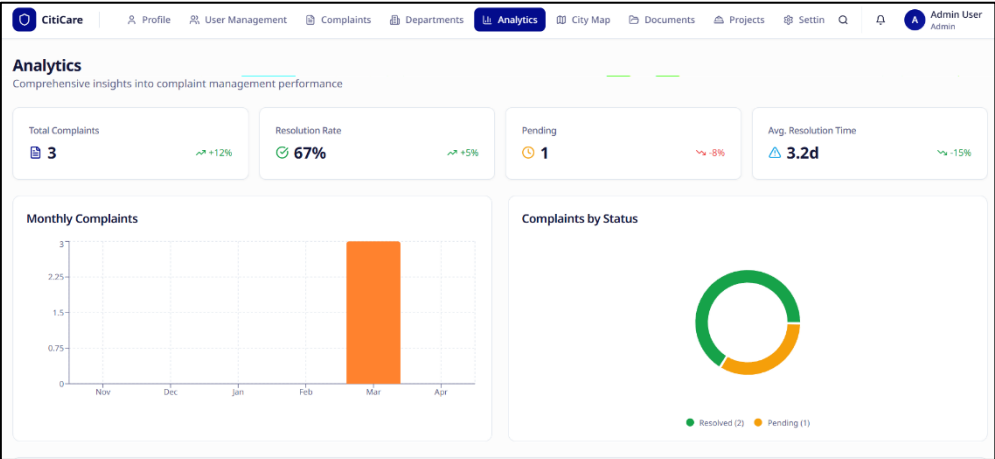
7.9 User Management



7.10 Admin Complaint Dashboard



7.11 All Departments



7.12 Complaints Analytics

7.2 Explanation of functionality achieved

The CitiCare system successfully achieves the following functionalities:

- Provides a centralized platform for citizens to report civic issues
- Enables real-time complaint tracking and transparency
- Supports image upload for better issue understanding
- Allows users to upvote complaints, enabling priority-based handling
- Provides department-specific dashboards for efficient workflow management
- Enables budget estimation and approval by the admin
- Integrates map-based location tracking for accurate complaint reporting
- Ensures secure authentication and role-based access control

The system effectively bridges the gap between citizens and municipal authorities, improving communication, efficiency, and service quality.

CHALLENGES FACED

8.1 Development challenges encountered

During the development of the CitiCare system, several challenges were faced while implementing different modules of the application.

- **Database Design Complexity**
Designing an efficient database structure for users, complaints, votes, and budget modules required careful planning to avoid redundancy and ensure scalability.
- **Role-Based Access Control**
Implementing different user roles such as Citizen, Department Admin, and Municipal Head and restricting access accordingly was challenging.
- **API Integration Issues**
Connecting the frontend with backend APIs and handling asynchronous requests required proper error handling and response management.
- **Image Upload Handling**
Managing image uploads using Multer, including file validation and storage, was a challenging task.
- **Complaint Workflow Management**
Maintaining proper complaint status flow (Pending → Processing → Solved) across multiple modules required synchronization between components.
- **Map Integration**
Integrating location services and handling latitude and longitude data accurately required additional effort.

8.2 Solutions or workarounds used

The following solutions were implemented to overcome the challenges:

- **Structured Database Schema**
Used Mongoose to define proper schemas and relationships between collections.
- **JWT Authentication & Middleware**
Implemented secure authentication using JWT and role-based middleware for access control.
- **Efficient API Handling**
Used Axios for API communication and implemented proper error handling mechanisms.
- **File Upload Management**
Used Multer middleware for handling image uploads and storing file paths securely.
- **Defined Workflow Logic**
Implemented clear status transitions and ensured consistent updates across system modules.
- **Map API Integration**
Used map APIs to capture and store location coordinates effectively.

CONCLUSION AND FUTURE SCOPE

9.1 Summary of the project achievements

The **CitiCare – Smart Citizen Complaint Management System** was successfully designed and developed to address the challenges faced in traditional municipal complaint handling systems.

- The system provides a **centralized platform** for citizens to report civic issues efficiently.
- It improves **communication between citizens and municipal authorities** by enabling real-time complaint tracking.
- The implementation of **role-based access control** ensures secure and organized system usage for citizens, departments, and administrators.
- Features such as **image upload, location tracking, and upvote-based priority system** enhance the effectiveness of complaint management.
- The system ensures **transparency and accountability** by allowing users to monitor complaint progress and view resolution proof.
- Overall, CitiCare successfully demonstrates how modern web technologies can be used to improve urban governance and public service delivery.

9.2 Potential areas for improvement or future features

Although the CitiCare system provides a comprehensive solution, several enhancements can be implemented in the future to improve its functionality:

- **Mobile Application Development**
Develop a mobile app version to increase accessibility and user engagement.
- **Real-Time Notifications**
Integrate SMS or push notifications to inform users about complaint updates instantly.
- **AI-Based Issue Detection**
Implement AI to automatically detect and categorize issues from uploaded images.
- **Advanced Analytics Dashboard**
Provide detailed insights such as area-wise issue trends and department performance.
- **Integration with Government Systems**
Connect with official municipal systems for real-time processing and faster resolution.
- **Multi-language Support**
Support multiple regional languages to make the system accessible to a wider audience.
- **Public Awareness Features**
Add features like alerts, announcements, and community updates.

REFERENCES

- Express.js Documentation – <https://expressjs.com>
- MongoDB Documentation – <https://www.mongodb.com/docs>
- Mongoose Documentation – <https://mongoosejs.com>
- React.js Documentation – <https://react.dev>
- Axios Documentation – <https://axios-http.com>
- Node.js Documentation – <https://nodejs.org>
- Postman API Testing Tool – <https://www.postman.com>
- Google Chrome Developer Tools
– <https://developer.chrome.com/docs/devtools>
- REST API Design Guide – <https://restfulapi.net>
- Tailwind CSS Documentation – <https://tailwindcss.com/docs>